

ElaScout API

Backend REST API para ElaScout. Node.js + Express + TypeScript + Firebase Admin SDK.

Requisitos

- Node.js >= 20
- Proyecto Firebase creado con Firestore y Auth habilitados
- Service Account Key (JSON) descargada desde Firebase Console

Setup local

1. Instalar dependencias

Desde la raíz del monorepo:

```
npm install
```

2. Obtener credenciales de Firebase

1. Ir a [Firebase Console](#) → tu proyecto (`redde-elascout`)
2. Configuración del proyecto (engranaje) → **Cuentas de servicio**
3. Click en “**Generar nueva clave privada**” → se descarga un JSON
4. Del JSON descargado, copiar los valores de `project_id`, `client_email` y `private_key` al `.env`

Importante: No subas el JSON ni el `.env` a git (ambos están en `.gitignore`).

3. Configurar variables de entorno

Copiar el archivo de ejemplo y completar:

```
cp .env.example .env
```

Editar `.env` con los valores del JSON de la service account:

```
# Server
PORT=4000
NODE_ENV=development

# Firebase Admin SDK
FIREBASE_PROJECT_ID=redde-elascout
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-xxxxx@redde-elascout.iam.gserviceaccount.com
FIREBASE_PRIVATE_KEY="-----BEGIN RSA PRIVATE KEY-----\nTU_CLAVE_AQUI\n-----END RSA PRIVATE KEY-----"

# CORS
CORS_ORIGIN=http://localhost:3000
```

Variable	Descripción	Requerida
PORT	Puerto del servidor	No (default: 4000)
NODE_ENV	Entorno de ejecución	No (default: development)
FIREBASE_PROJECT_ID	ID del proyecto Firebase	Si
FIREBASE_CLIENT_EMAIL	Email de la service account	Recomendada
FIREBASE_PRIVATE_KEY	Clave privada RSA (entre comillas dobles)	Recomendada
CORS_ORIGIN	URL del frontend permitida	No (default: http://localhost:3000)

Si no se configuran `FIREBASE_CLIENT_EMAIL` y `FIREBASE_PRIVATE_KEY`, el backend intentará usar solo `FIREBASE_PROJECT_ID` con Application Default Credentials (requiere `firebase login` previo).

4. Iniciar en modo desarrollo

```
# Desde la raíz del monorepo
npx turbo run dev --filter=elascout-api

# O directamente desde esta carpeta
npm run dev
```

El servidor arranca en `http://localhost:4000`. Deberías ver:

```
[firebase] Initialized with project: redde-elascout
[elascout-api] Running on http://localhost:4000
```

5. Verificar que funciona

```
curl http://localhost:4000/api/health
```

Respuesta esperada:

```
{ "status": "ok", "service": "elascout-api" }
```

Scripts disponibles

Script	Comando	Descripción
dev	npm run dev	Inicia el servidor con hot-reload (tsx watch)
build	npm run build	Compila TypeScript a dist/
start	npm run start	Ejecuta la versión compilada
lint	npm run lint	Ejecuta ESLint sobre src/

Endpoints

Método	Ruta	Auth	Descripción
GET	/api/health	No	Health check
POST	/api/auth/verify	No	Verifica token de Google y auto-crea usuario

Estructura

```
src/
├── index.ts           # Entry point, Express app
├── config/
│   └── firebase.ts    # Firebase Admin SDK init
├── middleware/
│   └── auth.middleware.ts # Verificación de token JWT
├── routes/
│   └── auth.routes.ts   # Rutas de autenticación
├── controllers/
│   └── auth.controller.ts # Lógica de request/response
├── services/
│   └── user.service.ts   # Lógica de negocio (Firestore)
├── models/             # Interfaces TypeScript
└── utils/              # Utilidades compartidas
```

Troubleshooting

Error: FIREBASE_PROJECT_ID is required → Asegúrate de tener FIREBASE_PROJECT_ID en el .env.

Error: Credential implementation provided to initializeApp() via the “credential” property failed to fetch a valid Google OAuth2 access token → Verifica que FIREBASE_CLIENT_EMAIL y

`FIREBASE_PRIVATE_KEY` sean correctos. La clave privada debe estar entre comillas dobles y los saltos de línea como `\n`.

CORS errors desde el frontend → Verifica que `CORS_ORIGIN` coincida con la URL exacta del frontend (incluyendo puerto).